서울시립대학교 · 전자전기컴퓨터설계실험II

LAB6 실험 전 보고서

FPGA6 / MCU2 (STM32_2) — 서보모터 · SRAM 읽기·쓰기 · DAC · 컴퓨터와 FPGA 통신 (제공 예제 2개 · 과제5 · 과제6 · 과제7)

이름
형종희

학번
2023440145

조
C조

작성일
2026-10-19

실험일
2026-10-26

FPGA 보드
Combo II-DLD S75 · xc7s75fgga484-1

MCU 보드
NUCLEO-G474RE

도구
Vivado 2026.1 (XSim) · Icarus Verilog 12.0 · STM32Cube (GCC 14.3.1)

📦 GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
LAB6 설계 소스 커밋	`618512b` — FPGA 4개 회로의 RTL · 테스트벤치 · 장치 모델 · Icarus/XSim 결과와 파형(VCD) · 합성/구현 보고서, STM32 예제·과제5~7 코드와 PC 모의 실행 파일
LAB6 경로	`LAB6/circuits/LAB6-01~04/` , `LAB6/stm32/` , `LAB6/LAB6_실험전보고서.html` · `.pdf`
실험 전 보고서 태그	`lab6-prelab` — 이 보고서(HTML · PDF)가 들어간 커밋. 최종 제출 태그 `lab6-submit` 은 실험 후 보고서 커밋에 따로 건다.
STM32 기반 프로젝트	`Glaysia/ece2_474` 태그 `v1.1.0` = 커밋 `dd94f01` . 학생이 바꾼 파일은 `Core/Src/ece2_474_routine.c` 하나이며, 예제·과제별 사본을 위 저장소에 보관.

0. 사전 결과 요약

LAB6은 10/26 수업(STM32_2 · 컴퓨터와 FPGA 통신)의 실습이다. **FPGA 개별 실습 3개**(서보모터 → SRAM → DAC)와 **STM32 제공 예제 2개, 직접 작성 과제 3개**(과제5 · 과제6 · 과제7)를 준비했다. 과제6·7에서 FPGA가 할 일(받은 문자 되돌려 보내기, 문자로 LED 제어)은 FPGA 회로 하나 `uart_cmd_top` 으로 묶었다.

모든 FPGA 회로는 **Icarus와 Vivado XSim 두 시뮬레이터**에서 테스트벤치(TB)의 자동 검사를 통과했다. 수업 TB 외에, 실제 보드에서 문제가 될 만한 부분을 보려고 **보드용 TB를 따로 만들었다**. 서보는 버튼 떨림을 넣어 보고, SRAM·DAC는 데이터시트의 시간 조건을 넣은 칩 모델에 연결했다. 그 결과 **수업 코드에서 보드 문제 3가지**를 찾아 보드용 최상위에서 고쳤다(아래 경고 상자).

#	실험	검증	결과	자원 / 빌드
1	서보모터 (FPGA)	Icarus · XSim 자동 검사 2종	PASS 20 ms 프레임 · 1/1.5/2 ms 폭 · 한계 · 버튼 떨림	교재 LUT 452 / 보드 LUT 411 · **WNS +12.86 ns**
2	SRAM 읽기·쓰기 (FPGA)	Icarus · XSim 자동 검사 2종	PASS 16주소 쓰기·읽기 · 버스 Z · 동시 요청	교재 LUT 4 · IOB 55 / 보드 LUT 12 · IOB 58
3	DAC (FPGA)	Icarus · XSim 자동 검사 2종	PASS 260/300회 쓰기 · 255→0 · 채널 · AD7302 시간 조건	보드 LUT 22 · FF 62 · **WNS +14.85 ns**
4	STM32_2 제공 예제 2개	빌드 + PC 모의 실행	PASS 문자 되돌려 출력 · UART3 전송 결과 0	실험 당일 실행
5	STM32_2 과제5 · PC→MCU→PC	빌드 + PC 모의 실행	PASS 1 · A · a → 0x31 · 0x41 · 0x61	FLASH text 27,740 B
6	STM32_2 과제6 · PC→MCU→FPGA→PC	빌드 + MCU 출력을 FPGA TB에 넣은 공동 시뮬레이션	PASS 3바이트 모두 FPGA가 되돌려 보냄	text 27,784 B · FPGA LUT 142 · **WNS +14.75 ns**
7	STM32_2 과제7 · UART 명령으로 LED	빌드 + 공동 시뮬레이션	PASS 1 반복 = 켜짐 유지, 2 반복 = 매번 반전, 0x01은 무시	text 27,820 B · (FPGA는 과제6과 같은 회로)

수업 코드에서 찾은 보드 문제 3가지 (LAB5 ADC처럼, 수업 FPGA 코드는 "보드 시연 미수행" 상태로 배포됨)

1. **서보 · 타이밍 위반**: `servo_pwm` 의 폭 계산 `pulse_us*(CLK_HZ/1000)/1000` 은 32비트 곱셈과 1000 나눗셈을 한 클록(20 ns) 안에 끝내야 한다. 50 MHz로 구현하면 **WNS = −0.609 ns**(20.6 ns가 필요). 1 µs가 정확히 50클록이므로 `pulse_us × 50` 으로 바꾸어 **WNS +12.86 ns**, LUT 782 → 411.
2. **DAC · 쓰기 펄스 폭 여유 0**: 쓰기 LOW 펄스가 1클록 = 20.0 ns인데 AD7302의 최소 폭도 20 ns다. 보드용은 **2클록(40 ns, 여유 20 ns)**으로 늘렸다.
3. **SRAM · 데이터 홀드 경쟁**: 쓰기 버튼을 떼는 순간 WE가 HIGH가 되는 것과 FPGA가 데이터선을 놓는(Z) 것이 **같은 순간**이다. SRAM은 쓰기가 끝나는 에지에서 데이터를 저장하고 홀드 시간 조건은 0 ns라서, 여유가 전혀 없다. 지연 0 시뮬레이션에서는 **17번 쓰기 모두 Z가 저장**됐다(두 시뮬레이터 같음). 보드용은 50 MHz 클록으로 데이터를 **WE 상승 뒤 2클록(40 ns) 더** 유지한다 → 경쟁 0, 버스 충돌 0.

수업 TB는 이 셋을 잡지 못한다. 서보 TB는 100 kHz 느린 클록을 쓰고(타이밍과 무관), DAC TB는 펄스 폭이 "9 ns 이상"인지만 보며, SRAM TB의 모델은 쓰기가 **시작**할 때 저장한다(실제 칩은 **끝**날 때 저장).

공통 사항

- **설계 소스 출처**: 교재 FPGA 모듈과 TB는 수업 저장소 `docs/FPGA_additional.md` (v1.1.0) 5 · 6 · 8번을 그대로 꺼내 썼다. 수업 TB는 고치지 않았고, 파형 저장은 별도 파일 `sim/dump.sv` 로 했다(XSim은 `-d XSIM` 으로 파일 이름만 바꿈). 보드용 최상위(`*_board_top`)와 장치 모델, 보드 TB는 직접 작성했다.
- **두 시뮬레이터 비교**: 7개 TB의 출력 문구와 측정 숫자(펄스 폭, 경쟁 횟수, UART 수신 시각)가 Icarus와 XSim에서 **모두 같았다**. 단, XSim은 Icarus보다 엄격해서 "선언 전 사용"과 "시간 0의 X→0 에지"를 드러냈다(고찰 2).
- **클록**: FPGA 50 MHz(B6, 주기 20 ns). MCU는 HSI 16 MHz → PLL 100 MHz, UART2(PC 연결) · UART3(FPGA 연결) 모두 9600 bps 8N1.

실험 당일 확인할 것 — ① **서보 출력 핀**과 **SRAM 핀 37개**(주소 18 · 데이터 16 · CS/OE/WE, UB/LB 유무)는 교재 핀표 사진을 보고 채운다(지금은 서보 = 임시 E8, SRAM = 합성까지만). ② **DAC 칩 이름**: 핀 이름(A/B · CS · WR · LDAC)이 AD7302와 같아 그 데이터시트로 검토했다. 보드의 칩 인쇄로 확인한다. ③ **MCU→FPGA 점퍼**: PC10 → 확장 GPIO BDIO_00(F8, PORT ENABLE ON), GND ↔ GND. 커넥터 위치는 장비 핀맵으로 확인한다. ④ 비트스트림(DAC · UART 명령 · 서보)은 이미 만들어 두었으므로 핀이 다를 때만 `setpin6` → `rebuild6` .

목차

1. 실험 1 · 서보모터
2. 실험 2 · SRAM
3. 실험 3 · DAC
4. 실험 4 · STM32_2 제공 예제
5. 실험 5 · 과제5
6. 실험 6 · 과제6
7. 실험 7 · 과제7
8. 고찰
9. 참고문헌

1. 실험 1 · 서보모터 제어 (servo_ctrl)

① 목적

RC 서보모터는 **20 ms마다 한 번 오는 펄스의 HIGH 길이**로 각도를 정한다. 버튼(왼쪽 · 오른쪽)을 누를 때마다 펄스 폭을 0.1 ms씩 바꾸어 서보 각도가 단계적으로 움직이는 회로를 만들고, 폭이 1.0 ~ 2.0 ms 범위를 벗어나지 않는지 확인한다.

② 이론 · 설계 계산

서보 신호 읽는 법. 서보는 손목시계처럼 "20 ms마다 한 번" 오는 신호를 기다리고, 그 신호가 HIGH로 머문 시간을 잰다. 1.5 ms면 가운데, 1.0 ms와 2.0 ms면 양쪽 끝 쪽으로 돌아간다(정확한 각도는 서보 기종마다 다르다). 그래서 회로는 두 가지만 하면 된다. ① 20 ms 길이의 프레임을 계속 반복하고 ② 프레임 시작부터 원하는 폭만큼만 HIGH를 낸다.

$$N_{\text{frame}} = \frac{f_{clk}}{50} = \frac{50\,\text{MHz}}{50} = 1\,000\,000 \text{ 클록 } (20\ \text{ms}), \qquad N_{\text{width}} = t_{\mu s} \times \frac{f_{clk}}{10^6} = t_{\mu s} \times 50$$

pulse_us	폭	50 MHz 클록 수	TB(100 kHz) 클록 수	LED5~8 (위치 2진수)
1000	1.0 ms	50,000	100	0000 (0)
1500 (리셋)	1.5 ms	75,000	150	0101 (5)
2000	2.0 ms	100,000	200	1010 (10)

모듈 구성. `servo_position`은 왼쪽 펄스에 −100 µs, 오른쪽 펄스에 +100 µs(1000 ~ 2000 사이에서만, 둘 다 들어오면 무시). `servo_pwm`은 0 ~ 999,999를 세는 카운터이고 `count < width_cycles`일 때 HIGH를 낸다. 새 폭은 **프레임 경계에서만** 반영해서, 펄스 도중 폭이 바뀌어 잘리는 일이 없다.

버튼 처리(보드용). `left_pulse` · `right_pulse`는 "버튼 한 번 = 한 클록 펄스"여야 한다. 손가락으로 누른 버튼은 수 ms 동안 접점이 떨려(바운스) 0/1을 여러 번 오가므로, 그대로 넣으면 한 번 눌러도 여러 칸 움직인다. 보드용 최상위 `servo_board_top`에 `btn_onepulse`를 넣었다: 2단 플립플롭 동기화 → 10 ms(500,000 클록) 동안 같은 값이면 인정 → 0→1로 바뀌는 순간 한 클록 펄스.

타이밍 문제와 해결. 클록 한 칸(20 ns) 안에 끝나야 하는 계산이 너무 길면 FPGA가 제때 답을 못 낸다. 시험 시간 20분인데 25분짜리 문제를 낸 셈이다. 수업 식 `pulse_us*(CLK_HZ/1000)/1000`은 32비트 곱셈 뒤 1000 나눗셈이라 경로가 20.609 ns로 0.609 ns 늦었다(**WNS = −0.609 ns**). 50 MHz에서는 1 µs = 정확히 50클록이므로

$$\frac{t_{\mu s} \times 50\,000}{1000} = t_{\mu s} \times 50 = t_{\mu s} \times (32 + 16 + 2)$$

로 바꾸면 덧셈 세 번이면 된다. 보드용 `servo_pwm_board`는 클록이 1 MHz의 배수일 때만 이 식을 쓰고, TB의 느린 클록에서는 원래 식을 쓴다(`generate`). 1.0 ~ 2.0 ms의 11개 값 모두에서 두 식의 결과가 같음을 보드 TB에서 확인했다.

③ 포트 · 보드 매핑

포트	방향	장치	FPGA 핀	상태
clk_50mhz	입력	메인 클록 50 MHz	B6	확인됨(LAB4·5)
rst_p	입력	DIP SW1 (ON = 리셋)	Y1	확인됨
btn_left	입력	키패드 4 (누르면 HIGH)	N1	확인됨
btn_right	입력	키패드 6	N6	확인됨
servo	출력	서보 신호선	E8 (임시 · BDIO_01)	당일 교재 핀표로 교체
led[7]	출력	LED1 = 서보 신호(흐리게 켜짐)	L4	확인됨
led[3:0]	출력	LED5~8 = 위치 0~10 (2진수)	M3 M1 N5 …	확인됨

④ RTL

servo_ctrl.v (교재 최상위)

```
`timescale 1ns/1ps
module servo_ctrl #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,left_pulse,right_pulse,
    output wire servo
);
```

```
    wire [31:0] pulse_us;
    servo_position u_position(clk_50mhz,rst_p,left_pulse,right_pulse,pulse_us);
    servo_pwm #(.CLK_HZ(CLK_HZ)) u_pwm(clk_50mhz,rst_p,pulse_us,servo);
endmodule
```

servo_position.v

```
`timescale 1ns/1ps
// 버튼은 동기화·디바운스 후 한 클록 길이의 펄스로 전달합니다.
module servo_position(
    input wire clk_50mhz,rst_p,left_pulse,right_pulse,
    output reg [31:0] pulse_us
);
    always @(posedge clk_50mhz)
        if(rst_p) pulse_us<=1500;
        else if(left_pulse && !right_pulse && pulse_us>1000) pulse_us<=pulse_us-100;
        else if(right_pulse && !left_pulse && pulse_us<2000) pulse_us<=pulse_us+100;
endmodule
```

servo_pwm.v (교재 · 타이밍 위반)

```
`timescale 1ns/1ps
module servo_pwm #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,
    input wire [31:0] pulse_us,
    output wire servo
);
    localparam FRAME_CYCLES=CLK_HZ/50; // 20 ms
    reg [31:0] count;
    reg [31:0] width_cycles;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; width_cycles<=(CLK_HZ/1000)*3/2; end
        else if(count==FRAME_CYCLES-1) begin
            count<=0;
            // 프레임 경계에서만 폭을 바꿔 중간 펄스가 잘리지 않게 합니다.
            width_cycles<=pulse_us*(CLK_HZ/1000)/1000;
        end
        else count<=count+1;
    end
    assign servo=!rst_p && (count<width_cycles);
endmodule
```

servo_pwm_board.v (보드용 · ×50)

```
`timescale 1ns/1ps
// 보드용 servo_pwm. 수업 servo_pwm과 동작은 같고, 폭 계산식만 바꿨다.
// 수업식 pulse_us*(CLK_HZ/1000)/1000 은 32비트 곱셈 + 1000 나눗셈이 한 클록(20 ns) 안에 끝나야 해서
// 50 MHz 구현에서 WNS = -0.609 ns(타이밍 위반). CLK_HZ가 1 MHz의 배수이면 1 µs당 클록 수가 정수이므로
// pulse_us * (CLK_HZ/1_000_000) 한 번의 상수 곱(50 = 32+16+2, 덧셈 3개)으로 같은 값을 얻는다.
module servo_pwm_board #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,
    input wire [31:0] pulse_us,
    output wire servo
);
    localparam FRAME_CYCLES=CLK_HZ/50; // 20 ms
    wire [31:0] width_next;
    generate
        if (CLK_HZ % 1_000_000 == 0) begin : g_exact
            assign width_next = pulse_us * (CLK_HZ/1_000_000);       // 보드(50 MHz): ×50
        end else begin : g_general
            assign width_next = pulse_us * (CLK_HZ/1000) / 1000;     // TB용 느린 클록(100 kHz 등)
        end
    endgenerate
    reg [31:0] count;
    reg [31:0] width_cycles;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; width_cycles<=(CLK_HZ/1000)*3/2; end
        else if(count==FRAME_CYCLES-1) begin
            count<=0;
            width_cycles<=width_next;       // 프레임 경계에서만 폭을 바꿔 중간 펄스가 잘리지 않게 한다
        end
        else count<=count+1;
    end
    assign servo=!rst_p && (count<width_cycles);
endmodule
```

btn_onepulse.v

```
`timescale 1ns/1ps
// 키패드 버튼(누르면 HIGH) -> 2단 동기화 -> 디바운스 -> 누를 때 한 클록 펄스.
// CNT 클록 동안 같은 값이 유지되어야 새 상태로 인정한다(50 MHz에서 500000 = 10 ms).
module btn_onepulse #(parameter integer CNT = 500_000)(
    input  wire clk, rst_p, btn,
    output reg  pulse
);
    (* ASYNC_REG = "TRUE" *) reg [1:0] sync;
    reg stable;
    reg [31:0] cnt;
```

```
    always @(posedge clk) begin
        if (rst_p) begin
            sync <= 2'b00; stable <= 1'b0; cnt <= 0; pulse <= 1'b0;
        end else begin
            sync  <= {sync[0], btn};
            pulse <= 1'b0;
            if (sync[1] == stable) cnt <= 0;              // 변화 없음: 카운터 비움
            else if (cnt == CNT-1) begin                  // 바뀐 값이 CNT 클록 유지됨
                cnt <= 0; stable <= sync[1];
                if (sync[1]) pulse <= 1'b1;               // 눌림(0->1)에서만 펄스
            end else cnt <= cnt + 1;
        end
    end
endmodule
```

servo_board_top.v (보드용 최상위)

```
`timescale 1ns/1ps
// 보드 시연용 최상위: 키패드 버튼 -> btn_onepulse -> servo_position -> servo_pwm.
// 수업 servo_ctrl과 같은 두 모듈을 연결하고, 현재 위치(0~10)를 LED로 보여 준다.
module servo_board_top #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer DEBOUNCE_MS = 10
)(
    input  wire clk_50mhz, rst_p, btn_left, btn_right,
    output wire servo,
    output wire [7:0] led
);
    wire left_pulse, right_pulse;
    wire [31:0] pulse_us;
    btn_onepulse #(.CNT(CLK_HZ/1000*DEBOUNCE_MS)) u_left (clk_50mhz, rst_p, btn_left,  left_pulse);
    btn_onepulse #(.CNT(CLK_HZ/1000*DEBOUNCE_MS)) u_right(clk_50mhz, rst_p, btn_right, right_pulse);
    servo_position u_position(clk_50mhz, rst_p, left_pulse, right_pulse, pulse_us);
    servo_pwm_board #(.CLK_HZ(CLK_HZ)) u_pwm(clk_50mhz, rst_p, pulse_us, servo);   // 50 MHz 타이밍을 맞춘 보드용
    wire [31:0] step = (pulse_us - 32'd1000) / 32'd100;   // 1000 us -> 0 ... 2000 us -> 10
    assign led = {servo, 3'b000, step[3:0]};              // LED1 = 서보 신호, LED5~8 = 위치(2진수)
endmodule
```

⑤ 시뮬레이션

수업 TB(`sim/tb.sv` , CLK_HZ = 100 kHz로 축소 → 프레임 2000클록)는 리셋 직후 폭 150클록(1.5 ms) → 왼쪽 8번(한계 1000에서 멈춤) → 100클록 → 오른쪽 15번(한계 2000) → 200클록 → 두 버튼 동시 → 무시 → 리셋을 검사한다. 두 시뮬레이터 모두 `PASS servo: 20ms frame, 1/1.5/2ms pulse, limits, reset` (종료 100,036 ns).

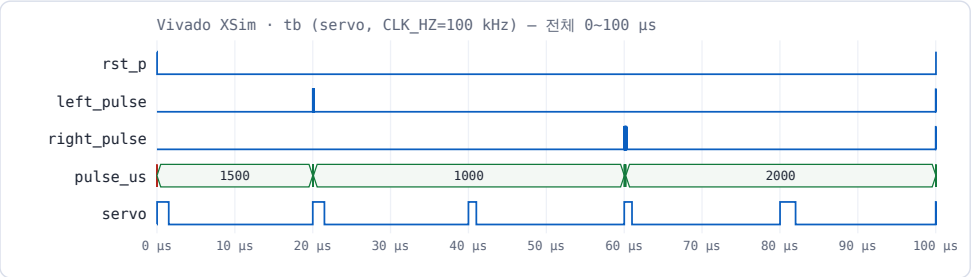


그림 1. 수업 TB 전체. pulse_us가 1500 → 1000(왼쪽 한계) → 2000(오른쪽 한계) → 리셋 1500으로 바뀌고, servo HIGH 폭이 다음 프레임부터 따라 바뀐다. 프레임 1개 = 2000클록 = 20 µs(TB 축소 시간).

보드 TB(`sim/tb_board.sv`)는 `servo_board_top` 에 **떨리는 버튼**을 넣는다. 누를 때 30번, 뗄 때 20번 3클록 간격으로 0/1을 뒤집은 뒤 6 ms 동안 누르고 있어도 **한 번에 한 칸만** 움직이는지 본다(디바운스 1 ms = 100클록으로 축소). 왼쪽 1번 → 1.4 ms, 오른쪽 3번 → 1.7 ms를 프레임 단위로 세어 확인하고, LED 위치 값(5 → 7)도 검사한다. 이어서 50 MHz 설정의 ×50 경로가 수업 식과 같은 값을 내는지 1000 ~ 2000 µs 11개 값을 비교한다. 결과 `PASS servo_board_top: bounce -> 1 pulse per press, 1.4/1.7 ms, LED step, x50 width = class formula (11 values)` .

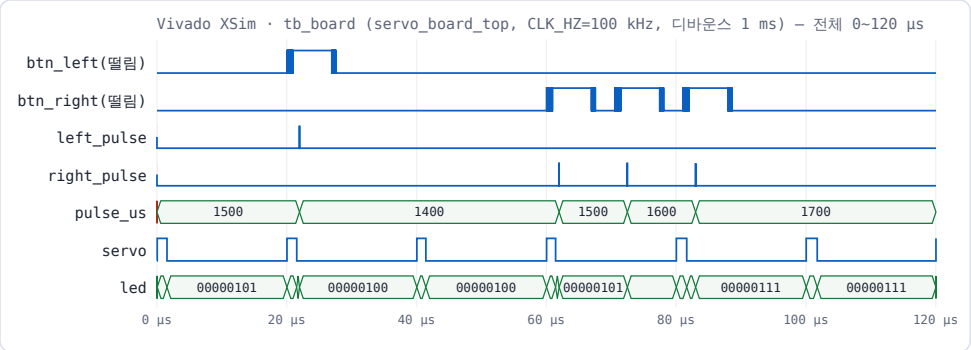


그림 2. 보드 TB 전체. 떨림 50번이 들어간 버튼 한 번에 left_pulse/right_pulse가 정확히 하나씩만 나오고(총 4개), pulse_us가 1500 → 1400 → 1500 → 1600 → 1700으로 한 칸씩 움직인다.

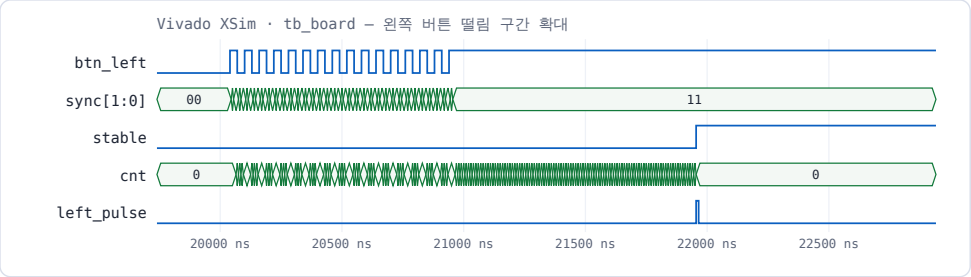


그림 3. 떨림 구간 확대. 입력이 3클록마다 바뀌는 동안 카운터 cnt는 매번 0으로 돌아가고, 입력이 100클록 동안 1로 머문 뒤에야 stable이 1이 되며 그 순간 left_pulse가 한 클록 나온다.

⑥ 합성 · 구현 결과

최상위	단계	LUT	FF	IOB	WNS / WHS
servo_ctrl (교재)	합성	452	73	5	—
servo_board_top + 교재 servo_pwm	구현(배치·배선)	782	119	13	**−0.609** / +0.215 ns
servo_board_top + servo_pwm_board	구현 · 비트스트림	411	125	13	**+12.860** / +0.115 ns

×1000 나눗셈이 없어지자 LUT가 절반으로 줄고 여유가 12.86 ns 생겼다. 비트스트림 `LAB6_bit/LAB6-01_servo.bit` 은 서보 핀 E8(임시)로 만들어 두었다.

⑦ 보드 실습 계획

1. 교재 핀표에서 서보 신호선 핀을 확인 → `setpin6 1 servo <핀>` → `rebuild6 1` (약 3분, 끝나면 자동으로 보드에 올림). 서보 전원(5 V)과 GND 연결은 교재 그림대로.
2. DIP SW1을 ON → OFF(리셋 해제): 서보가 가운데(1.5 ms), LED5~8 = 0101.
3. 오실로스코프(1 V/칸, 5 ms/칸)를 서보 신호에 연결: 주기 20.0 ms, 양의 펄스 폭 1.50 ms 확인. 키패드 4를 5번 → 1.00 ms(LED 0000), 6을 10번 → 2.00 ms(LED 1010). 한계에서 더 눌러도 그대로인지 확인.
4. 버튼을 한 번 누를 때 정확히 한 칸(0.1 ms)만 바뀌는지 확인(디바운스 동작). 영상 하나에 서보 움직임 + 오실로스코프 폭 변화를 담는다.

2. 실험 2 · SRAM 읽기·쓰기 (sram)

① 목적

외부 SRAM 칩의 주소 0 ~ 15에 DIP 스위치로 정한 4비트 값을 쓰고, 같은 주소에서 다시 읽어 LED로 확인한다. FPGA와 SRAM이 **같은 데이터선(16비트 양방향 버스)**을 번갈아 쓰는 방법, 즉 쓸 때는 FPGA가 구동하고 읽을 때는 FPGA가 손을 떼는(Z) 방법을 익힌다.

② 이론 · 설계 계산

양방향 버스는 한 줄짜리 무전기. 데이터선 16개는 FPGA와 SRAM이 함께 쓴다. 둘이 동시에 말하면(둘 다 구동) 한쪽은 1, 다른 쪽은 0을 내서 전류가 흐르는 **버스 충돌**이 생긴다. 그래서 FPGA는 쓰기 때만 값을 내고, 그 밖에는 출력을 **Z(고임피던스, 손 떼기)**로 둔다: `assign ram_data = drive_data ? {12'b0,data_in} : 16'bz;`

제어 신호는 모두 LOW 활성(이름 끝 `_n`의 뜻). 수업 `sram_bus_control`은 순수 조합회로다.

ctrl_w	ctrl_r	동작	CS	OE	WE	FPGA 데이터선	LED
0	0	쉼	1	1	1	Z	0
1	0	쓰기	0	1	**0**	data_in 구동	0
0	1	읽기	0	**0**	1	Z (SRAM이 구동)	ram_data[3:0]
1	1	동시 요청 → 무시	1	1	1	Z	0

쓰기는 "끝나는 순간"에 저장된다. 비동기 SRAM의 쓰기 구간은 CS · WE(· UB/LB)가 모두 LOW인 동안이고, **그중 하나가 먼저 HIGH가 되는 순간** 데이터선의 값이 저장된다. 그래서 데이터는 그 순간 직전 t_{SD} 동안, 직후 t_{HD} 동안 그대로 있어야 한다(ISSI IS61WV25616 -10 등급).

기호	뜻	최소/최대	교재 sram	보드용 sram_board_top
t_{PWE1}	WE LOW 폭	≥ 8 ns	버튼 누름(µs~ms)	같음
t_{SD}	쓰기 끝 전 데이터 유지	≥ 6 ns	버튼 누름 동안 유지	같음
t_{HD}	쓰기 끝 후 데이터 유지	**≥ 0 ns**	**0 ns (WE↑와 Z가 동시)**	**40 ns (2클록)**
t_{AA} / t_{DOE}	주소·OE → 읽은 값 확정	≤ 10 / 4.5 ns	사람 속도라 문제 없음	같음
t_{HZOE}	OE↑ → SRAM이 손 뗌	≤ 4 ns	—	—

왜 t_{HD} = 0이 위험한가. 조건이 "0 ns 이상"이라 언뜻 쉬워 보이지만, 교재 회로에서는 `drive_data` 하나가 WE와 데이터선 출력 허용(Z 여부)을 **동시에** 바꾼다. 실제 칩에서 WE가 0.5 ns라도 늦게 올라가면, SRAM은 이미 Z가 된(값이 정해지지 않은) 선을 저장한다. 결과는 FPGA 핀의 지연 차이와 배선에 남은 전하에 달려 있어 **되는 보드도 있고 안 되는 보드도 있는** 설계다. 보드용은 50 MHz 클록으로 ① 버튼을 2단 동기화하고 ② WE를 올린 뒤에도 데이터를 `HOLD_CYCLES = 2` 클록 더 구동한 다음 Z로 놓는다. 읽기는 데이터를 놓은 뒤에만 시작한다.

UB/LB. 16비트 SRAM은 위·아래 바이트 선택(UB · LB, LOW 활성)이 있는 경우가 많다. 이 핀이 FPGA에 연결되어 있다면 LOW로 두어야 쓰기가 된다. 보드용 최상위는 `ram_ub = ram_lb = 0` 을 낸다(교재 핀표에 없으면 보드에서 GND 고정된 것이므로 그 두 포트를 뺀다).

③ 포트 · 보드 매핑

포트	방향	장치	FPGA 핀	상태
ctrl_w	입력	키패드 1 = 쓰기	K4	확인됨
ctrl_r	입력	키패드 2 = 읽기	N8	확인됨
address[3:0]	입력	DIP SW1~4	Y1 W3 U2 T1	확인됨
data_in[3:0]	입력	DIP SW5~8	W4 W1 V4 U4	확인됨
led_out[7:0]	출력	LED1~8 (읽은 값 = LED5~8)	L4 … N5	확인됨
clk_50mhz	입력	보드용만	B6	확인됨
ram_cs / oe / we	출력	SRAM 제어(LOW 활성)	?	교재 핀표
ram_ub / ram_lb	출력	바이트 선택(있으면)	?	교재 핀표
ram_address[17:0]	출력	SRAM 주소	?	교재 핀표
ram_data[15:0]	양방향	SRAM 데이터	?	교재 핀표 · 방향 대조

④ RTL

sram.v (교재 최상위)

```
`timescale 1ns/1ps
module sram(
    input wire ctrl_w,ctrl_r,
    input wire [3:0] address,data_in,
    output wire ram_cs,ram_oe,ram_we,
    output wire [17:0] ram_address,
    inout wire [15:0] ram_data,
    output wire [7:0] led_out
);
    wire drive_data,read_data;
    sram_bus_control u_control(ctrl_w,ctrl_r,drive_data,read_data,ram_cs,ram_oe,ram_we);
    assign ram_address={14'b0,address};
```

```
    assign ram_data=drive_data ? {12'b0,data_in} : 16'bz;
    assign led_out=read_data ? {4'b0,ram_data[3:0]} : 8'b0;
endmodule
```

sram_bus_control.v

```
`timescale 1ns/1ps
// SRAM 버스는 읽을 때 FPGA가 구동하지 않고 Z 상태로 놓습니다.
// ctrl_w와 ctrl_r가 동시에 들어오면 둘 다 수행하지 않습니다.
module sram_bus_control(
    input wire ctrl_w,ctrl_r,
    output wire drive_data,read_data,
    output wire ram_cs,ram_oe,ram_we
);
    assign drive_data=ctrl_w && !ctrl_r;
    assign read_data=ctrl_r && !ctrl_w;
    assign ram_cs=!(drive_data || read_data);
    assign ram_oe=!read_data;
    assign ram_we=!drive_data;
endmodule
```

sram_board_top.v (보드용 최상위)

```
`timescale 1ns/1ps
// 보드 시연용 SRAM 최상위.
// 수업 sram은 조합회로라서 쓰기 버튼을 떼는 순간 WE 상승과 데이터 버스 해제(Z)가 동시에 일어난다
// -> SRAM 데이터 홀드 tHD = 0 ns 조건과 여유 0의 경쟁. 여기서는 50 MHz 클록으로
//   (1) 버튼을 2단 동기화하고, (2) 쓰기가 끝난 뒤 데이터를 HOLD_CYCLES 클록 더 구동한 다음 Z로 놓는다.
// 동시 요청 무시, 미사용 시 Z, LED 표시는 수업 sram과 같다. UB/LB(바이트 선택, LOW 활성)는 LOW 고정.
module sram_board_top #(parameter integer HOLD_CYCLES = 2)(
    input  wire clk_50mhz,
    input  wire ctrl_w, ctrl_r,
    input  wire [3:0] address, data_in,
    output wire ram_cs, ram_oe, ram_we,
    output wire ram_ub, ram_lb,
    output wire [17:0] ram_address,
    inout  wire [15:0] ram_data,
    output wire [7:0] led_out
);
    (* ASYNC_REG = "TRUE" *) reg [1:0] w_sync = 2'b00, r_sync = 2'b00;
    reg [3:0] hold = 0;
    always @(posedge clk_50mhz) begin
        w_sync <= {w_sync[0], ctrl_w};
        r_sync <= {r_sync[0], ctrl_r};
    end
    wire drive_req, read_req, cs_unused, oe_unused, we_unused;
    // 수업 버스 제어 모듈을 동기화된 버튼으로 그대로 사용(동시 요청 무시 규칙 유지)
    sram_bus_control u_control(w_sync[1], r_sync[1], drive_req, read_req, cs_unused, oe_unused,
we_unused);
    always @(posedge clk_50mhz)
        if (drive_req) hold <= HOLD_CYCLES;
        else if (hold != 0) hold <= hold - 1'b1;
    wire drive_data = drive_req || (hold != 0);          // 쓰기 끝난 뒤에도 데이터 유지
    wire read_data  = read_req && (hold == 0);           // 데이터 해제 후에만 읽기
    assign ram_we = !drive_req;
    assign ram_oe = !read_data;
    assign ram_cs = !(drive_data || read_data);
    assign ram_ub = 1'b0;
    assign ram_lb = 1'b0;
    assign ram_address = {14'b0, address};
    assign ram_data = drive_data ? {12'b0, data_in} : 16'bz;
    assign led_out = read_data ? {4'b0, ram_data[3:0]} : 8'b0;
endmodule
```

async_sram_model.sv (보드 TB용 SRAM 모델 · ISSI -10 등급 시간 조건)

```
`timescale 1ns/1ps
// 256K x 16 비동기 SRAM 모델(10 ns급, IS61WV25616 계열 대표값). 읽기 경로 지연과 쓰기 타이밍 검사를 넣었다.
//  tAA 10 / tDOE 4.5 / tHZOE 4 (읽기),  tWP 8 / tSD 6 / tHD 0 / tSA 0 / tHA 0 (쓰기)
module async_sram_model(
    input wire ce_n, oe_n, we_n, ub_n, lb_n,
    input wire [17:0] a,
    inout wire [15:0] dq
);
    reg [15:0] mem [0:15];
    reg [15:0] dout; reg drive = 0;
    realtime t_we_fall = 0, t_addr = 0, t_dq = 0;
    integer violations = 0, writes = 0, contention = 0, hold_races = 0;
    wire read_en = !ce_n && !oe_n && we_n;
    // 쓰기 구간 = CE와 WE가 모두 LOW인 동안. 둘 중 먼저 HIGH가 되는 순간 데이터를 저장한다(데이터시트 정의).
    wire wr_active = !ce_n && !we_n;
    assign dq = drive ? dout : 16'bz;
    always @(a)  begin
        t_addr = $realtime;
        if (wr_active) begin violations = violations + 1; $display("%t SRAM violation: address changed
during write", $realtime); end
    end
    always @(dq) t_dq = $realtime;
    always @(read_en or a) begin
        if (read_en) begin #4.5; if (read_en) begin dout = 16'bx; drive = 1; #5.5; dout = mem[a[3:0]]; end
end
```

```
        else begin #4; drive = 0; end
    end
    reg armed = 0;   // 시작 순간 X->0 같은 초기 에지는 쓰기로 세지 않는다(실제 0->1->0 구간만)
    always @(posedge wr_active) if (wr_active === 1'b1) begin t_we_fall = $realtime; armed = 1; end
    always @(negedge wr_active) if (armed && wr_active === 1'b0) begin
        armed = 0;
        if ($realtime - t_we_fall < 8.0) begin violations = violations + 1; $display("%t SRAM violation: tWP", $realtime); end
        if (^dq === 1'bx) begin                          // 쓰기 끝과 같은 순간 버스가 Z/X -> tHD=0 경쟁
            hold_races = hold_races + 1;
        end else if ($realtime - t_dq < 6.0) begin violations = violations + 1; $display("%t SRAM violation: tSD", $realtime); end
        if (ub_n !== 1'b0 || lb_n !== 1'b0) begin violations = violations + 1; $display("%t SRAM violation: byte enable high", $realtime); end
        mem[a[3:0]] = dq; writes = writes + 1;
    end
    // 충돌 검사: SRAM이 확정된 값을 내보내는 중인데 버스 값이 다르면 FPGA도 구동 중인 것(충돌).
    always @(dq or drive or dout) #0.1 if (drive && (^dout !== 1'bx) && dq !== dout) begin
        contention = contention + 1; $display("%t SRAM bus contention dq=%h dout=%h", $realtime, dq, dout); end
endmodule
```

⑤ 시뮬레이션

수업 TB는 16개 주소에 15−i를 쓰고(쓰기 중 버스 값 · 주소 검사), 다시 읽어 LED와 버스를 확인한 뒤, 두 버튼 동시 → 버스 Z · CS/OE/WE 모두 1을 검사한다. 두 시뮬레이터 모두 `PASS SRAM: 16 addresses, bus release, simultaneous request` (종료 164 ns).

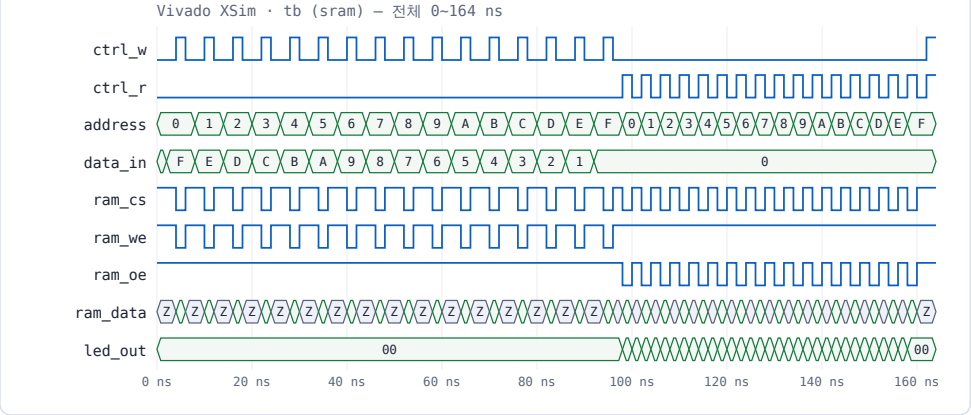


그림 4. 수업 TB 전체. 앞 96 ns는 쓰기 16번(WE 펄스, 버스 = F, E, … 0), 다음 64 ns는 읽기 16번(OE 펄스, LED = F, E, … 0), 끝은 동시 요청(모두 1, 버스 Z).

보드 TB(`sim/tb_board.sv`)는 사람 손 속도(DIP 설정 1 µs → 버튼 2 µs)로 **교재 sram과 보드용 sram_board_top을 각각 SRAM 모델에 연결**해 같은 입력을 준다. 모델은 쓰기 끝 에지에 저장하고, t_{PWE} · t_{SD} 위반, 쓰기 중 주소 변화, UB/LB, 버스 충돌(SRAM이 값을 내는데 버스 값이 다름), 그리고 **쓰기 끝 순간 버스가 Z/X인 경우(t_{HD} 경쟁)**를 센다. "쓰기를 누른 채 읽기 누름 → 둘 다 무시(버스 Z) → 쓰기 뗌 → 읽기"도 확인한다.

```
CLASS sram     : writes=17, tHD=0 races=17, wrong reads=16/16
BOARD sram_top : writes=17, tHD=0 races=0, violations=0, contention=0
PASS SRAM board: 17 writes, 17 reads, overlap idle, data held after WE rise, no bus contention
$finish called at time : 120100 ns : File "C:/Users/UOS/Documents/lab6/lab6_02_sram/sim/tb_board.sv" Line 36
```

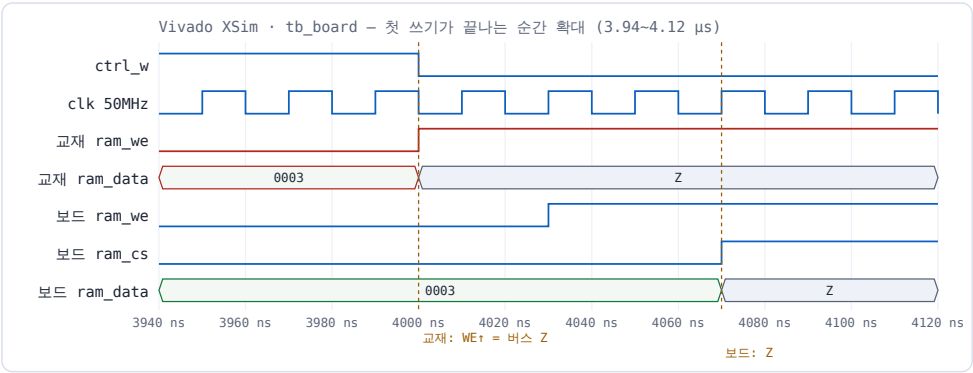


그림 5. 쓰기 끝 확대. 교재 회로는 4.000 µs에 WE 상승과 버스 Z가 같은 순간이다(경쟁). 보드용은 버튼 동기화로 30 ns 늦게 4.030 µs에 WE가 오르고, 데이터는 40 ns 더 유지한 뒤 4.070 µs에 놓는다(t_{HD} 여유 40 ns).

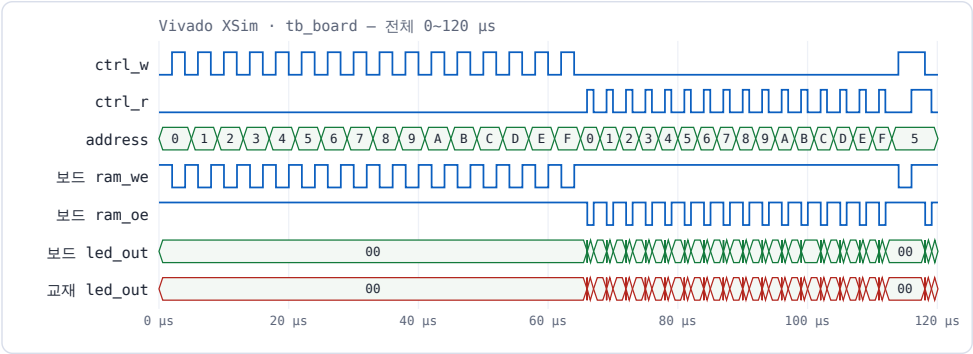


그림 6. 보드 TB 전체. 뒤쪽 읽기 16번에서 보드용 LED는 쓴 값(3, A, 1, 8, …)을 정확히 돌려주고, 교재 회로는 Z를 저장해 LED 아래 4비트가 Z로 읽힌다(지연 0 시뮬레이션의 결과). 맨 끝은 "쓰기 누른 채 읽기" 구간.

이 결과를 어디까지 믿을까 — 지연 0 시뮬레이션에서 "같은 순간"은 시뮬레이터가 정한 처리 순서로 승부가 난다. 두 시뮬레이터 모두 Z가 먼저 처리돼 교재 회로가 16/16 틀렸지만, **실제 보드에서는 핀 지연과 배선 전하 때문에 동작할 수도 있다.** 이 결과가 말하는 것은 "교재 회로는 동작이 보장되지 않는다(여유 0)"이고, 보드용은 어느 순서로 처리돼도 40 ns 여유가 있다는 점이다.

⑥ 합성 결과 (SRAM 핀 없이 합성까지)

최상위	LUT	FF	IOB	비고
sram (교재)	4	0	55	입력 10 + 출력 29 + 양방향 16
sram_board_top	12	8	58	+ clk · UB · LB, 동기화 FF 4 + 홀드 카운터 4

SRAM 핀을 모르므로 구현·비트스트림은 당일 교재 핀표를 받은 뒤 만든다(약 3분).

⑦ 보드 실습 계획

1. 교재 SRAM 핀표 사진 → XDC의 `ram_*` 37줄(UB/LB 포함 시 39줄) 작성, 데이터선은 **신호 이름 + 방향** 단위로 대조 → `build6 2` → `prog6 2`. UB/LB가 표에 없으면 최상위를 `sram`으로 바꾸지 않고 보드용에서 두 포트만 지운다(동기화 · 홀드 유지).
2. 쓰기: DIP SW1~4 = 주소, SW5~8 = 값 → 키패드 1을 눌렀다 뗀다. 주소 0~15에 서로 다른 값을 차례로 쓴다.
3. 읽기: 주소만 바꾸고 키패드 2를 누르고 있는 동안 LED5~8 = 쓴 값인지 확인. 두 키를 함께 누르면 LED가 꺼지는지(무시) 확인.
4. (선택) 교재 `sram`을 최상위로 한 비트스트림으로도 같은 시험을 해 보고, 실제 보드에서 경쟁이 드러나는지 기록한다.

3. 실험 3 · DAC (dac)

① 목적

8비트 병렬 DAC 칩에 0, 1, 2, ..., 255, 0, ...을 차례로 써서 계단 모양 톱니파를 만들고, 오실로스코프로 디지털 값에 따라 아날로그 전압이 달라지는 것을 확인한다. 데이터를 먼저 놓고 쓰기 펄스로 "저장"하는 병렬 버스 쓰기 순서와, 칩 데이터시트의 시간 조건을 지키는 법을 익힌다.

② 이론 · 설계 계산

DAC는 숫자를 전압으로 바꾸는 계량컵. 8비트 값 D를 주면 기준 전압 V_{ref}를 256칸으로 나눈 것 중 D칸 만큼의 전압을 낸다.

$$V_{out} = V_{ref} \cdot \frac{D}{256}, \qquad \Delta V = \frac{V_{ref}}{256}$$

예를 들어 $V_{ref} = 2.5\ \mathrm{V}$이면 한 칸은 9.77 mV이고, $D = 255$일 때 2.490 V가 최대다.

값을 바꾸는 속도. `ramp8`은 50 MHz에서 500,000클록마다(10 ms) 값을 1 올린다. 256칸을 다 오르는 데

$$T_{ramp} = 256 \times \frac{500\,000}{50\ \mathrm{MHz}} = 256 \times 10\ \mathrm{ms} = 2.56\ \mathrm{s} \quad (f = 0.39\ \mathrm{Hz})$$

가 걸린다. 오실로스코프에서는 500 ms/칸 정도로 봐야 톱니 한 주기가 보이고, 칸(9.8 mV 계단)은 확대해야 보인다. 실제 V_{ref}는 교재 회로도로 확인한다(보드 TB 모델은 2.5 V로 가정).

쓰기 순서. `dac_parallel8`은 `update` 클록에 데이터 · A/B 채널을 레지스터에 넣고 같은 클록에 WR을 LOW로 만든 뒤, 다음 클록에 WR을 HIGH로 되돌린다. CS와 LDAC는 항상 LOW이므로, 칩은 **WR이 올라가는 순간** 값을 받아 바로 출력을 바꾼다(LDAC LOW = 즉시 갱신).

기호 (AD7302)	뜻	최소	교재 dac (1클록)	보드용 dac_board_top (2클록)
t_5	WR LOW 폭	20 ns	**20.0 ns (여유 0)**	**40.0 ns (여유 20)**
t_6	데이터 셋업 (→ WR↑)	15 ns	20.0 ns	40.0 ns
t_7	데이터 홀드 (WR↑ →)	4.5 ns	≥ 140 ns (TB) / 10 ms (보드)	≥ 120 ns / 10 ms
t_1,t_2 / t_3,t_4	A/B · CS 셋업·홀드	0 ns	CS 상시 LOW	같음

여유 0이 왜 문제인가. FPGA 안의 20 ns는 정확하지만, 핀을 나와 칩에 닿기까지 상승·하강 에지의 지연이 조금씩 다르다. 하강보다 상승이 1 ns 빠르면 칩이 보는 펄스는 19 ns가 되어 조건을 어긴다. 그래서 보드용은 쓰기 펄스를 `WR_CYCLES = 2` 클록(40 ns)으로 늘렸다. 값은 10 ms마다 바뀌므로 20 ns 늘려도 동작에는 영향이 없다.

칩 이름 주의. 교재 핀표의 신호 이름(DAC_D7~D0, DAC_A_B, DAC_CSN, DAC_LDACN, DAC_WRN)이 Analog Devices AD7302(2채널 8비트 병렬 DAC)와 정확히 같아서 그 데이터시트로 검토했다. 보드에 실제로 붙은 칩의 인쇄를 당일 확인한다.

③ 포트 · 보드 매핑 (교재 p.625)

포트	방향	장치	FPGA 핀	상태
clk_50mhz	입력	메인 클록	B6 (교재 N8은 오기 → 키패드 2)	**정정 · 확인됨**
rst_p	입력	키패드 1	K4	**교재 포트표**
add_sel	입력	DIP SW1 (채널 A/B)	Y1	**교재 포트표**
dac_d[7:0]	출력	DAC_D7(MSB) ~ D0	B3 B1 C4 C2 D5 D2 E5 E3	**보드 미검증 · 방향 대조**
dac_a_b · dac_csn	출력	DAC_A_B · DAC_CSN	G2 · E1	**보드 미검증**
dac_ldacn · dac_wen	출력	DAC_LDACN · DAC_WRN	F4 · G4	**보드 미검증**
led_out[7:0]	출력	LED1~8 = dac_d	L4 … N5	**확인됨**

④ RTL

dac.v (교재 최상위)

```
`timescale 1ns/1ps
module dac #(parameter UPDATE_CYCLES=500_000)(
    input wire clk_50mhz,rst_p,add_sel,
    output wire dac_csn,dac_ldacn,dac_wen,dac_a_b,
    output wire [7:0] dac_d,led_out
);
    wire [7:0] value;
    wire update;
    ramp8 #(.UPDATE_CYCLES(UPDATE_CYCLES)) u_ramp(clk_50mhz,rst_p,value,update);
    dac_parallel8 u_bus(clk_50mhz,rst_p,update,add_sel,value,
        dac_csn,dac_ldacn,dac_wen,dac_a_b,dac_d);
```

```
    assign led_out=dac_d;
endmodule
```

ramp8.v

```
`timescale 1ns/1ps
module ramp8 #(parameter UPDATE_CYCLES=500_000)(
    input wire clk_50mhz,rst_p,
    output reg [7:0] value,
    output wire update
);
    reg [31:0] count;
    assign update=(count==UPDATE_CYCLES-1);
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; value<=0; end
        else if(update) begin count<=0; value<=value+1'b1; end
        else count<=count+1;
    end
endmodule
```

dac_parallel8.v (교재)

```
`timescale 1ns/1ps
// 데이터는 쓰기 LOW 펄스 전에 등록하고, 펄스 종료까지 유지합니다.
module dac_parallel8(
    input wire clk_50mhz,rst_p,update,channel,
    input wire [7:0] value,
    output wire cs_n,ldac_n,
    output reg we_n,a_b,
    output reg [7:0] data
);
    assign cs_n=0;
    assign ldac_n=0;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin we_n<=1; a_b<=0; data<=0; end
        else begin
            we_n<=1;
            if(update) begin data<=value; a_b<=channel; we_n<=0; end
        end
    end
endmodule
```

dac_board_top.v (보드용 · WR 2클록)

```
`timescale 1ns/1ps
// 보드 시연용 DAC 최상위. 수업 dac와 같은 ramp8을 쓰되, 쓰기 LOW 폭을 WR_CYCLES 클록으로 늘린다.
// AD7302 계열 병렬 DAC의 WR 최소 폭은 20 ns -> 50 MHz 1클록(20 ns)은 여유 0이므로 2클록(40 ns) 사용.
module dac_board_top #(
    parameter integer UPDATE_CYCLES = 500_000,
    parameter integer WR_CYCLES = 2
)(
    input  wire clk_50mhz, rst_p, add_sel,
    output wire dac_csn, dac_ldacn,
    output reg  dac_wen, dac_a_b,
    output reg  [7:0] dac_d,
    output wire [7:0] led_out
);
    wire [7:0] value;
    wire update;
    reg  [3:0] wr_left;
    ramp8 #(.UPDATE_CYCLES(UPDATE_CYCLES)) u_ramp(clk_50mhz, rst_p, value, update);
    assign dac_csn = 1'b0;          // 칩 선택 상시
    assign dac_ldacn = 1'b0;        // LDAC LOW: WR 상승 에지에서 바로 출력 갱신
    always @(posedge clk_50mhz) begin
        if (rst_p) begin dac_wen <= 1'b1; dac_a_b <= 1'b0; dac_d <= 8'd0; wr_left <= 0; end
        else if (update) begin
            dac_d <= value; dac_a_b <= add_sel; dac_wen <= 1'b0; wr_left <= WR_CYCLES-1;
        end else if (wr_left != 0) wr_left <= wr_left - 1'b1;
        else dac_wen <= 1'b1;
    end
    assign led_out = dac_d;
endmodule
```

ad7302_model.sv (보드 TB용 DAC 모델)

```
`timescale 1ns/1ps
// AD7302(이중 8비트 병렬 DAC) 디지털 인터페이스 + 출력 모델.
// 데이터시트 B등급(VDD 2.7~5.5 V) 최소값: t5 WR 폭 20 ns, t6 데이터 셋업 15 ns, t7 데이터 홀드 4.5 ns,
// t1/t2 A/B 셋업·홀드 0 ns, t3/t4 CS 셋업·홀드 0 ns. LDAC LOW이면 WR 상승 에지에서 DAC 레지스터 갱신.
module ad7302_model #(parameter real VREF = 2.5)(
    input wire cs_n, wr_n, ldac_n, a_b,
    input wire [7:0] d
);
    real vout_a = 0.0, vout_b = 0.0;
    realtime t_wr_fall = 0, t_d_change = 0, t_ab_change = 0, t_wr_rise = -1000;
    realtime min_wr = 1e9, min_su = 1e9, min_hold = 1e9;
    integer writes = 0, violations = 0;
    reg armed = 0;   // X->1 같은 리셋 초기 에지는 쓰기로 세지 않는다
```

```
    always @(d)   begin
        t_d_change = $realtime;
        if ($realtime - t_wr_rise < 4.5 && writes > 0 && $realtime > t_wr_rise) begin
            violations = violations + 1; $display("%t AD7302 data hold violation", $realtime); end
        if ($realtime - t_wr_rise < min_hold && writes > 0) min_hold = $realtime - t_wr_rise;
    end
    always @(a_b) t_ab_change = $realtime;
    always @(negedge wr_n) if (wr_n === 1'b0) begin
        t_wr_fall = $realtime; armed = 1;
        if (cs_n !== 1'b0) begin violations = violations + 1; $display("%t AD7302 CS not low at WR fall",
$realtime); end
    end
    always @(posedge wr_n) if (cs_n === 1'b0 && armed && wr_n === 1'b1) begin
        armed = 0;
        t_wr_rise = $realtime;
        if ($realtime - t_wr_fall < min_wr) min_wr = $realtime - t_wr_fall;
        if ($realtime - t_d_change < min_su) min_su = $realtime - t_d_change;
        if ($realtime - t_wr_fall < 20.0) begin violations = violations + 1; $display("%t AD7302 WR pulse
%0.1f ns < 20", $realtime, $realtime - t_wr_fall); end
        if ($realtime - t_d_change < 15.0) begin violations = violations + 1; $display("%t AD7302 data
setup %0.1f ns < 15", $realtime, $realtime - t_d_change); end
        if (ldac_n === 1'b0) begin
            if (a_b) vout_b = VREF * d / 256.0; else vout_a = VREF * d / 256.0;
        end
        writes = writes + 1;
    end
endmodule
```

⑤ 시뮬레이션

수업 TB(UPDATE_CYCLES = 4, 10 ns 클록)는 쓰기마다 ① WR LOW 폭 9 ns(TB 기준) ② 펄스 동안 데이터 유지 ③ 값 = 쓰기 횟수 mod 256 ④ LED = 데이터 ⑤ CS · LDAC = 0, A/B = 채널을 검사한다. 130번째 쓰기 뒤 채널을 1로 바꾸고 260번(255 → 0 넘어감 포함) 뒤 리셋한다. 두 시뮬레이터 모두 `PASS DAC: 260 writes, wraparound, channel, data hold, reset`(종료 10,436 ns).

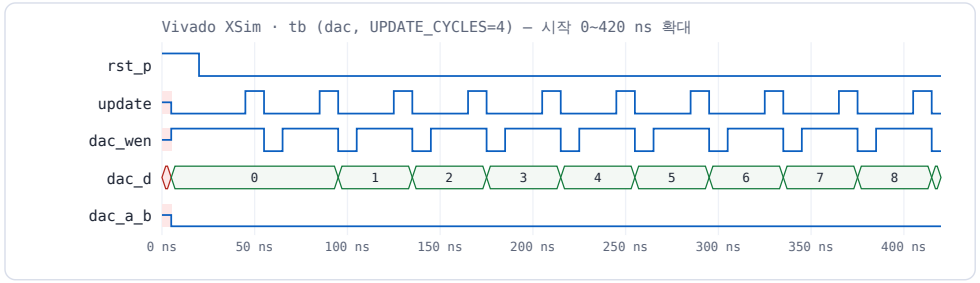


그림 7. 시작 부분. update가 4클록마다 한 번 나오고, 그 클록에 dac_d가 다음 값으로 바뀌며 dac_wen이 한 클록 LOW가 된다.

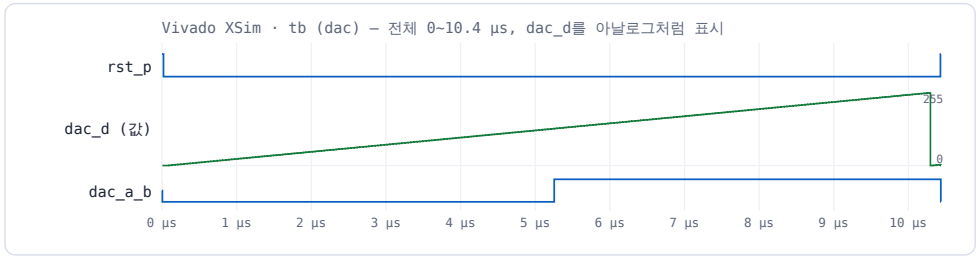


그림 8. 전체. 0 → 255 계단을 다 오른 뒤 0으로 돌아가고(넘어감), 130번째 쓰기에서 채널이 B로 바뀐 뒤 260번째 쓰기 후 리셋.

보드 TB(`sim/tb_board.sv`)는 **실제 50 MHz 주기(20 ns)**로 교재 dac와 보드용 dac_board_top을 각각 AD7302 모델에 연결하고, 300번 쓰는 동안 가장 짧았던 WR 폭 · 데이터 셋업 · 홀드를 잰다. 모델은 WR 상승에서 $V_{out} = 2.5\,\mathrm{V} \times D/256$을 갱신한다.

```
CLASS dac      : WR=20.0 ns (min 20 -> margin 0.0), setup=20.0 ns (min 15), hold>=140.0 ns, violations=0
BOARD dac_top  : WR=40.0 ns (min 20 -> margin 20.0), setup=40.0 ns (min 15), hold>=120.0 ns, violations=0
PASS DAC board: 300 writes, AD7302 timing OK, vout ramp 0..2.4902 V
$finish called at time : 48080 ns : File "C:/Users/UOS/Documents/lab6/lab6_03_dac/sim/tb_board.sv" Line 24
```

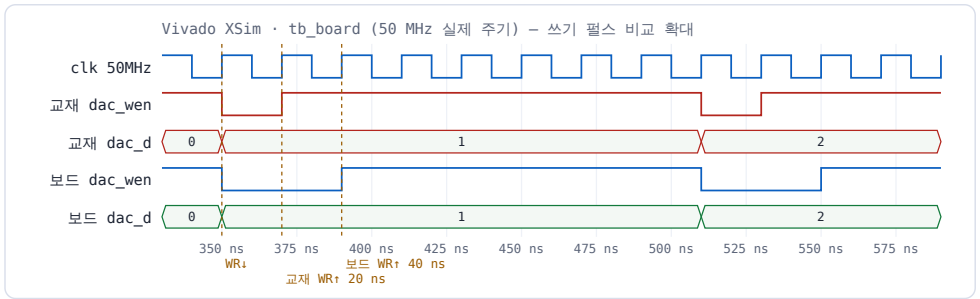


그림 9. 쓰기 펄스 비교. 교재 회로의 WR LOW는 정확히 1클록(20 ns, AD7302 최소값과 같음), 보드용은 2클록(40 ns).

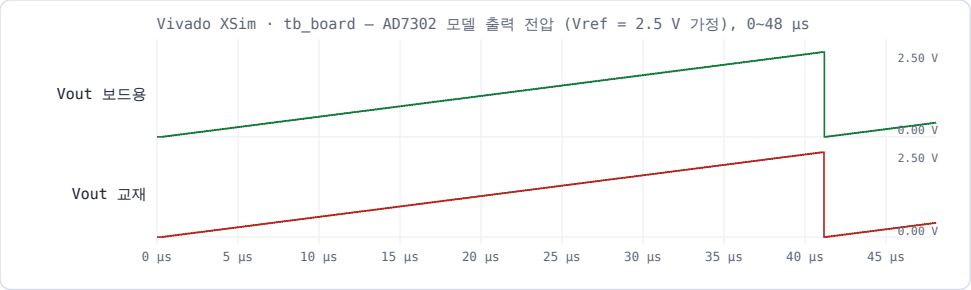


그림 10. 모델 출력. 0 V에서 2.490 V까지 9.77 mV씩 256계단을 오른 뒤 0 V로 떨어지는 톱니. 보드에서는 한 계단이 10 ms라 한 주기 2.56 s.

⑥ 합성 · 구현 결과

최상위	단계	LUT	FF	IOB	WNS / WHS
dac (교재)	합성	19	50	23	—
dac_board_top	합성	23	54	23	—
dac_board_top	구현 · 비트스트림	22	62	23	**+14.854** / +0.165 ns

FF 50개 = 카운터 32 + 값 8 + 데이터 8 + WR 1 + A/B 1. 구현 단계에서 FF가 62개로 는 것은 출력 핀 근처로 레지스터를 복제한 결과다. 비트스트림 `LAB6_bit/LAB6-03_dac.bit` 은 교재 핀표대로 만들어 두었다.

⑦ 보드 실습 계획

1. 교재 p.625 핀표와 회로도로 칩 이름 · V_{ref} · 출력 단자(VOUTA/VOUTB) 위치를 확인하고, XDC 신호 방향을 대조. 이상 없으면 `prog6 3` .
2. 오실로스코프를 VOUTA에(1 V/칸 또는 500 mV/칸, 500 ms/칸): 0 V → V_{ref}×255/256까지 2.56 s 톱니. LED1~8이 2진수로 세어 올라가는지 함께 촬영.
3. 시간축을 50 ms/칸으로 줄여 계단(10 ms 간격, 높이 V_{ref}/256) 확인, 커서로 한 칸 높이를 잰다.
4. DIP SW1(add_sel)을 ON으로 바꾸면 VOUTB가 움직이고 VOUTA는 마지막 값에 멈추는지 확인. 키패드 1(리셋)을 누르면 0 V.

4. 실험 4 · STM32_2 제공 예제 2개

① 목적

① PC의 Serial Monitor에서 보낸 문자를 MCU가 `getchar()` 로 받아 그대로 다시 출력하고, ② 받은 한 바이트를 UART3(PC10)로 FPGA에 보내 그 결과값(`HAL_OK` = 0)을 출력한다. 이후 과제 5 ~ 7의 바탕이 된다.

② 이론 · 설계 계산

getchar는 기다린다. `getchar()` → `_read()` → `__io_getchar()` → `HAL_UART_Receive(&huart2, &byte, 1, HAL_MAX_DELAY)` 로 이어진다. 문자가 올 때까지 이 함수 안에서 계속 기다리므로, 입력이 없을 때 화면이 조용한 것은 **멈춘(고장) 것이 아니라 기다리는 중**이다. 결과를 `int` 에 받는 이유는 0 ~ 255 문자 말고도 오류 표시 `EOF` (−1)를 구분해야 하기 때문이다.

자료형. `UART_HandleTypeDef` 는 UART 설정·상태를 묶은 구조체이고 `huart3` 은 그 변수다(`ece2_474.h` 에 extern 선언). `HAL_StatusTypeDef` 는 결과를 나타내는 enum: HAL_OK = 0, HAL_ERROR = 1, HAL_BUSY = 2, HAL_TIMEOUT = 3. `HAL_UART_Transmit(&huart3, &data, 1U, 100U)` 은 "huart3 장치로, data가 있는 주소에서, 1바이트를, 최대 100 ms 기다리며 보내라"는 뜻이다.

전송 결과 0의 의미. HAL_OK는 "MCU가 마지막 비트까지 내보냈다"는 뜻일 뿐, **FPGA가 받았다는 뜻이 아니다**(UART는 받는 쪽의 응답이 없다). 받았는지는 FPGA 출력으로 따로 확인해야 한다(과제6).

③ 포트 · 보드 매핑

MCU 핀	용도	연결
PA2 / PA3	UART2 TX/RX (printf · getchar)	보드 안에서 ST-LINK USB → PC의 VCP COM 포트
PC10	UART3 TX → FPGA	점퍼선 → FPGA 확장 GPIO RX (예제 ②, 과제6·7)
GND	공통 접지	MCU GND ↔ FPGA GND

④ 핵심 코드

예제 ① polling_routine 교체

```
/* STM32_2 제공 예제 1: PC에서 받은 문자를 그대로 다시 출력합니다. */
void polling_routine(void)
{
    int ch = getchar();
    if (ch != EOF)
    {
        printf("받은 문자: %c\r\n", ch);
    }
}
```

예제 ② polling_routine 교체

```
/* STM32_2 제공 예제 2: PC에서 받은 한 바이트를 UART3(PC10)로 FPGA에 보냅니다. */
void polling_routine(void)
{
    int ch = getchar();
    if (ch != EOF)
    {
        uint8_t data = (uint8_t)ch;
        HAL_StatusTypeDef result =
            HAL_UART_Transmit(&huart3, &data, 1U, 100U);
        printf("전송 결과: %d\r\n", (int)result);
    }
}
```

⑤ 시뮬레이션 (PC 모의 실행)

LAB5와 같은 방법이다. `main.h`를 PC용 가짜 헤더로 바꾸고(UART · TIM 핸들 구조체, HAL 상태 enum, 핀 이름), **저장소의 `ece2_474_routine.c`를 한 글자도 바꾸지 않고** gcc로 컴파일했다. 표준 입력이 Serial Monitor 역할을 하고, 가짜 `HAL_UART_Transmit`은 UART3로 나간 바이트를 `uart3_tx_*.hex` 파일에 기록한다. 이 파일은 과제6·7에서 FPGA TB의 입력이 된다.

예제 ① 입력 "Hi"

```
받은 문자: H
받은 문자: i
```

예제 ② 입력 "Z"

```
전송 결과: 0
```

UART3로 나간 바이트: 0x5A

⑥ 빌드 결과

예제는 과제5~7과 같은 프로젝트에서 실험 당일 `mcu6 ex1 -Flash`로 올린다(빌드 경로는 과제와 동일).

⑦ 보드 실습 계획

1. `mcu6 ex1 -Flash` → Serial Monitor(9600, 8N1, 줄바꿈 없음)에서 문자 전송 → "받은 문자: ...".
2. `mcu6 ex2 -Flash` → 문자 전송 → "전송 결과: 0". PC10에 오실로스코프를 대면 9600 bps 직렬 파형(비트 하나 104 µs)이 보인다.

5. 실험 5 · STM32_2 과제5 — PC → MCU → PC

① 목적

PC에서 보낸 문자 한 개를 MCU가 받아 **문자와 그 바이트 값(16진수)**을 함께 출력한다. 문자 `1` · `A` · `a` 가 **0x31 · 0x41 · 0x61**로 나오는지 확인한다.

② 이론 · 설계 계산

문자도 결국 숫자. 키보드의 글자는 ASCII 표에 따라 한 바이트 숫자로 바뀌어 전송된다. 숫자 글자 '0' ~ '9'는 0x30 ~ 0x39, 대문자는 0x41부터, 소문자는 대문자 + 0x20이다. 그래서 글자 '1'(0x31)과 숫자 1(0x01)은 완전히 다른 값이다.

문자	'0'	'1'	'2'	'A'	'a'	CR (\r)	LF (\n)	숫자 1
16진수	0x30	0x31	0x32	0x41	0x61	0x0D	0x0A	0x01
10진수	48	49	50	65	97	13	10	1

출력 형식. `printf("0x%02X", data)` 의 `%02X` 는 "16진수 대문자, 두 자리, 빈 자리는 0"이다. 화면에 찍히지 않는 제어 문자(0x20 미만, 0x7F 이상)는 글자 대신 "(제어 문자)"와 값만 출력한다. Serial Monitor의 줄바꿈 설정이 "CR+LF"이면 문자 뒤에 0x0D · 0x0A가 따라오는 것이 그대로 보여서, 매뉴얼이 줄바꿈을 **없음(None)**으로 하라는 이유를 확인할 수 있다.

③ 포트 · 보드 매핑

UART2(VCP)만 사용. FPGA 연결 없음.

④ 핵심 코드 (템플릿과 다른 부분)

```
/* STM32_2_과제5: PC -> MCU -> PC
 * 문자 한 개를 받아 '문자'와 바이트 값(16진수)을 VCP(UART2)로 출력합니다.
 * 예) 1 -> 0x31, A -> 0x41, a -> 0x61
 */
void polling_routine(void)
{
    int ch = getchar();              /* 입력이 올 때까지 여기서 기다림(멈춘 것이 아님) */
    if (ch != EOF)                   /* EOF(-1) = 수신 오류. 정상 문자만 처리 */
    {
        uint8_t data = (uint8_t)ch;     /* 0~255 한 바이트로 변환 */
        if (data >= 0x20U && data <= 0x7EU)
        {
            /* 화면에 찍히는 문자(공백~'~')는 문자와 값을 함께 출력 */
            printf("받은 문자: '%c'  바이트 값: 0x%02X\r\n", (char)data, (unsigned int)data);
        }
        else
        {
            /* 줄바꿈(0x0D, 0x0A) 같은 제어 문자는 값만 출력 */
            printf("받은 문자: (제어 문자)  바이트 값: 0x%02X\r\n", (unsigned int)data);
        }
    }
}
```

줄	설명
`int ch = getchar();`	문자 하나가 올 때까지 기다린다. EOF(−1)까지 담기 위해 int.
`uint8_t data = (uint8_t)ch;`	0 ~ 255 한 바이트로 바꾼다(부호 문제 방지).
`if (data >= 0x20U && data <= 0x7EU)`	공백(0x20) ~ 물결표(0x7E)는 화면에 찍히는 문자.
`0x%02X`	두 자리 16진수. 0x0D처럼 한 자리 값도 0을 채워 맞춘다.

⑤ 시뮬레이션 (PC 모의 실행)

입력 `1Aa\r\n` (줄바꿈 CR+LF로 보낸 경우를 흉내):

```
받은 문자: '1'  바이트 값: 0x31
받은 문자: 'A'  바이트 값: 0x41
받은 문자: 'a'  바이트 값: 0x61
받은 문자: (제어 문자)  바이트 값: 0x0D
받은 문자: (제어 문자)  바이트 값: 0x0A
```

⑥ 빌드 결과

과제	text	data	bss	결과
task5	27,740 B	104 B	2,536 B	BUILD OK · 경고 0

⑦ 보드 실습 계획

1. `mcu6 task5 -Flash` → Serial Monitor 9600 8N1, 줄바꿈 **없음** → `1`, `A`, `a`를 하나씩 보내 0x31 · 0x41 · 0x61 확인.
2. 줄바꿈을 CR+LF로 바꿔 보내 0x0D · 0x0A가 더 찍히는 것을 확인(차이 기록). PowerShell로는 `talk6 '1Aa'`.
3. 아무것도 보내지 않을 때 출력이 없는 것 = 기다리는 상태임을 확인(LD2 등 다른 동작이 없으므로 다시 문자를 보내면 바로 응답).

6. 실험 6 · STM32_2 과제6 — PC → MCU → FPGA → PC

① 목적

MCU가 PC에서 받은 한 바이트를 UART3(PC10)로 FPGA에 넘기고, FPGA는 받은 문자를 자기 USB-UART(F6)로 PC에 다시 내보낸다. PC에 생기는 **두 COM 포트**(MCU VCP, FPGA USB-UART)를 함께 열어 보낸 문자와 FPGA가 받은 문자가 같은지 비교한다.

② 이론 · 설계 계산

UART 한 글자 = 10칸 기차. 선이 쉬는 동안은 HIGH. 보낼 때는 시작 비트(LOW) 1칸 → 데이터 8칸(**낮은 비트부터**) → 정지 비트(HIGH) 1칸이다. 9600 bps면 한 칸이

$$t_{bit} = \frac{1}{9600} = 104.17\ \mu\text{s}, \qquad t_{byte} = 10\,t_{bit} = 1.042\ \text{ms}$$

이다. 시계선이 따로 없으므로, 받는 쪽은 자기 시계로 시작 비트의 하강 에지부터 시간을 재어 **각 칸의 한가운데**에서 값을 읽는다. FPGA(50 MHz)에서 한 칸의 클록 수는

$$DIV = \text{round}\left(\frac{50\,000\,000}{9600}\right) = 5208, \qquad f_{\text{실제}} = \frac{50\,\text{MHz}}{5208} = 9600.6\ \text{bps}$$

(오차 0.006 %)이고, 수신기는 하강 에지 뒤 $DIV/2$에서 시작 비트를 다시 확인한 다음 DIV마다 8번 읽는다. 10칸 동안 쌓이는 어긋남이 반 칸(5 %)보다 작으면 되므로 0.006 %는 충분하다.

FPGA 회로 `uart_cmd_top`. LAB3-07 UART 에코 회로(`uart.v`의 `uart_rx` · `uart_tx`)를 그대로 쓰고, ① 수신 입력을 보드 USB-UART(C6) 대신 **확장 GPIO 핀** `mcu_rxd`로 바꾸고, ② 송신기가 바쁠 때 다음 바이트가 오면 한 개를 보관했다가 보내도록 했다. 받은 바이트는 그대로 F6로 다시 나간다.

③ 포트 · 보드 매핑

신호	출발	도착	FPGA 핀	상태
mcu_rxd	MCU PC10 (UART3 TX)	FPGA 확장 GPIO	F8 = BDIO_00 (PORT ENABLE ON)	커넥터 위치 당일 확인
uart_txd	FPGA	보드 USB-UART → PC	F6	LAB3에서 확인
GND	MCU GND (CN6)	FPGA GND	—	점퍼선
rst_p	키패드 1	FPGA	K4	확인됨
clk_50mhz			B6	확인됨

MCU와 FPGA 모두 3.3 V 로직(LVCMOS33)이라 바로 연결할 수 있다. 응답을 MCU로 돌려보내는 선(FPGA TX → PC11)은 이번 과제에 필요 없다.

④ 핵심 코드

MCU · polling_routine

```
/* STM32_2_과제6: PC -> MCU -> FPGA -> PC
 * PC에서 받은 한 바이트를 UART3(PC10, 9600 8N1)로 FPGA에 그대로 전달합니다.
 * FPGA는 받은 문자를 자기 USB-UART로 PC에 다시 내보냅니다(두 COM 포트 비교).
 */
void polling_routine(void)
{
    int ch = getchar();
    if (ch != EOF)
    {
        uint8_t data = (uint8_t)ch;
        /* 1바이트 송신, 최대 100 ms 대기. 결과 0(HAL_OK)은 "MCU가 다 보냈다"는 뜻일 뿐,
         * FPGA가 받았는지는 FPGA 쪽 COM 포트 출력으로 따로 확인합니다. */
        HAL_StatusTypeDef result = HAL_UART_Transmit(&huart3, &data, 1U, 100U);
        if (data >= 0x20U && data <= 0x7EU)
        {
            printf("FPGA로 보냄: '%c' (0x%02X)  전송 결과: %d\r\n",
                   (char)data, (unsigned int)data, (int)result);
        }
        else
        {
            printf("FPGA로 보냄: (제어 문자) (0x%02X)  전송 결과: %d\r\n",
                   (unsigned int)data, (int)result);
        }
    }
}
```

FPGA · uart_cmd_top.v (과제6·7 공용)

```
`timescale 1ns/1ps
// LAB6 과제6·7 공용 FPGA 회로 (LAB3-07 UART 에코 회로 확장)
//  - mcu_rxd : MCU PC10(UART3 TX)에서 오는 직렬 입력. 확장 GPIO(BDIO) 핀으로 받는다.  ← LAB3의 C6에서 변경
//  - uart_txd: 받은 바이트를 그대로 보드 USB-UART(F6)로 PC에 출력 (과제6)
//  - led[7] = LED1: '0' 끄기, '1' 켜기, '2' 반전, 그 밖은 유지 (과제7)
//  - led[6:0] = LED2~8: 마지막으로 받은 바이트의 하위 7비트 ('1'=0x31 -> 011_0001, 숫자 1=0x01 -> 000_0001)
module uart_cmd_top #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer BAUD   = 9_600
)(
    input  wire clk_50mhz,
    input  wire rst_p,
    input  wire mcu_rxd,
    output wire uart_txd,
    output wire [7:0] led
);
    localparam integer DIV = (CLK_HZ + BAUD/2) / BAUD;   // 50 MHz / 9600 = 5208 클록/비트
    wire [7:0] rx_data; wire rx_valid, rx_framing_error;
    wire tx_ready;
    reg  tx_valid; reg [7:0] tx_data;
    reg  pending; reg [7:0] pending_data;                // 송신 중에 다음 바이트가 오면 한 개 보관
    reg  lamp; reg [7:0] last_data;

    uart_rx #(.DIV(DIV)) u_rx(.clk(clk_50mhz), .rst_p(rst_p), .rx(mcu_rxd),
                              .data(rx_data), .valid(rx_valid), .framing_error(rx_framing_error));
    uart_tx #(.DIV(DIV)) u_tx(.clk(clk_50mhz), .rst_p(rst_p), .valid(tx_valid), .data(tx_data),
                              .ready(tx_ready), .tx(uart_txd));

    always @(posedge clk_50mhz) begin
        if (rst_p) begin
            tx_valid <= 0; tx_data <= 0; pending <= 0; pending_data <= 0;
            lamp <= 0; last_data <= 0;
        end else begin
            tx_valid <= 0;
            // 과제7: 명령 해석 (문자 '0'=0x30, '1'=0x31, '2'=0x32)
            if (rx_valid) begin
                last_data <= rx_data;
                case (rx_data)
```

```
                    8'h30:   lamp <= 1'b0;
                    8'h31:   lamp <= 1'b1;
                    8'h32:   lamp <= ~lamp;
                    default: lamp <= lamp;
                endcase
            end
            // 과제6: 에코. 송신기가 비어 있으면 바로, 바쁘면 한 바이트 보관 후 전송
            if (rx_valid && !pending && !(tx_ready && !tx_valid)) begin
                pending <= 1; pending_data <= rx_data;
            end
            if (tx_ready && !tx_valid) begin
                if (pending) begin tx_data <= pending_data; tx_valid <= 1; pending <= rx_valid;
pending_data <= rx_data; end
                else if (rx_valid) begin tx_data <= rx_data; tx_valid <= 1; end
            end
        end
    end
    assign led = {lamp, last_data[6:0]};
endmodule
```

FPGA · uart.v (LAB3-07 그대로)

```
`timescale 1ns/1ps
module uart_rx #(parameter integer DIV = 5208)(
    input wire clk, rst_p, rx,
    output reg [7:0] data,
    output reg valid,
    output reg framing_error
);
    (* ASYNC_REG="TRUE" *) reg [1:0] sync;
    reg [1:0] state;
    integer timer;
    reg [2:0] bitno;
    reg [7:0] shift;
    always @(posedge clk) begin
        if (rst_p) begin
            sync<=3; state<=0; timer<=0; bitno<=0; shift<=0;
            data<=0; valid<=0; framing_error<=0;
        end else begin
            sync<={sync[0],rx}; valid<=0; framing_error<=0;
            case(state)
                0: if(!sync[1]) begin timer<=DIV/2-1; state<=1; end
                1: if(timer!=0) timer<=timer-1;
                    else if(sync[1]) state<=0;
                    else begin timer<=DIV-1; bitno<=0; state<=2; end
                2: if(timer!=0) timer<=timer-1;
                    else begin
                        shift[bitno]<=sync[1]; timer<=DIV-1;
                        if(bitno==7) state<=3; else bitno<=bitno+1'b1;
                    end
                3: if(timer!=0) timer<=timer-1;
                    else begin
                        if(sync[1]) begin data<=shift; valid<=1; end
                        else framing_error<=1;
                        state<=0;
                    end
            endcase
        end
    end
endmodule

module uart_tx #(parameter integer DIV = 5208)(
    input wire clk, rst_p, valid,
    input wire [7:0] data,
    output wire ready,
    output wire tx
);
    reg [9:0] shift;
    reg [3:0] remaining;
    integer timer;
    assign ready = (remaining == 0);
    assign tx = ready ? 1'b1 : shift[0];
    always @(posedge clk) begin
        if(rst_p) begin shift<=10'h3ff; remaining<=0; timer<=0; end
        else if(ready) begin
            if(valid) begin shift<={1'b1,data,1'b0}; remaining<=10; timer<=DIV-1; end
        end else if(timer!=0) timer<=timer-1;
        else begin timer<=DIV-1; shift<={1'b1,shift[9:1]}; remaining<=remaining-1'b1; end
    end
endmodule
```

⑤ 시뮬레이션 (MCU 모의 실행 → FPGA TB 공동 시뮬레이션)

MCU 모의 실행에 `1Aa`를 넣어 UART3로 나간 바이트(`31 41 61`)를 파일로 받고, FPGA TB가 그 파일을 읽어 **실제 속도(50 MHz, 비트당 104.17 µs)**로 `mcu_rxd`에 직렬 파형을 만든다. TB 안의 "PC 수신기"는 `uart_txd`를 비트 한가운데에서 읽어, 되돌아온 바이트가 보낸 바이트와 같은지 검사한다.

MCU VCP 출력 (모의)

```
FPGA로 보냄: '1' (0x31)  전송 결과: 0
FPGA로 보냄: 'A' (0x41)  전송 결과: 0
```

FPGA TB (Icarus · 과제6 입력 3바이트)

```
2000.2 us  PC(FPGA COM) <- 0x31   LED1=1
LED2~8=0110001
  3406.5 us  PC(FPGA COM) <- 0x41   LED1=1
```

```
FPGA로 보냄: 'a' (0x61)  전송 결과: 0
```

```
LED2~8=1000001
  4812.7 us  PC(FPGA COM) <- 0x61   LED1=1
LED2~8=1100001
PASS uart_cmd_top: 3 bytes echoed, LED1 follows
0/1/2 commands
sim/tb.sv:46: $finish called at 5489750009 (1ps)
```

XSim은 과제7 입력(10바이트, 실험 7 ⑤)으로 실행했고, 그 안에 과제6의 동작(받은 바이트를 그대로 되돌려 보냄)도 모두 검사된다.

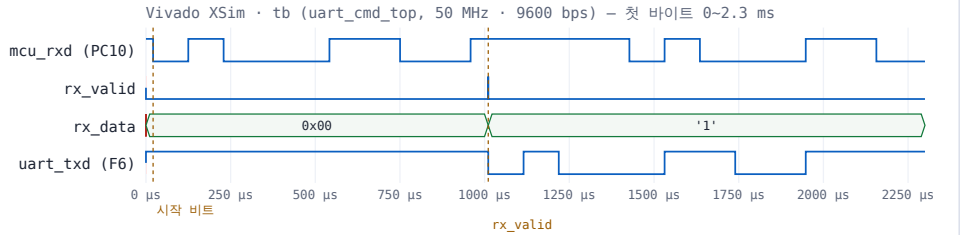


그림 11. MCU가 보낸 '1'(0x31) 한 글자: 시작 비트 LOW 뒤 1,0,0,0,1,1,0,0(낮은 비트부터) → 정지 비트. 정지 비트 중간에 rx_valid가 한 클록 나오고, 곧바로 uart_txd로 같은 모양이 다시 나간다.

⑥ 빌드 · 구현 결과

대상	LUT	FF	IOB	WNS / WHS	크기
uart_cmd_top (FPGA)	142	127	12	**+14.748** / +0.157 ns	비트스트림 LAB6-04_uart_cmd.bit
task6 (MCU)	—	—	—	—	text 27,784 B · data 104 B · bss 2,536 B

⑦ 보드 실습 계획

1. FPGA: PORT ENABLE ON, `prog6 4`. 점퍼: MCU PC10 → BDIO_00, GND ↔ GND.
2. MCU: `mcu6 task6 -Flash`. PC에 COM 포트 2개(VID_0483 = MCU, 나머지 = FPGA)가 보이는지 확인.
3. `talk6 '1Aa' -Both` : MCU 출력 "FPGA로 보냄 ... 전송 결과: 0"과 FPGA 포트 수신 "0x31 0x41 0x61"을 같은 화면에서 비교. 점퍼를 뽑고 보내 MCU는 여전히 0, FPGA는 0바이트임을 확인 → "전송 결과 0 ≠ 수신 성공".
4. LED2~8에 마지막 바이트의 아래 7비트가 표시되는 것도 함께 촬영.

7. 실험 7 · STM32_2 과제7 — UART 명령으로 FPGA LED 제어

① 목적

FPGA가 받은 문자를 **명령**으로 해석해 LED를 제어한다: `0` 끄기, `1` 켜기, `2` 반전, 그 밖의 문자는 유지. 특히 `1`을 반복하면 계속 켜져 있고, `2`를 반복하면 매번 바뀌는지 확인하고, 글자 '1'(0x31)과 숫자 1(0x01)을 구분한다.

② 이론 · 설계 계산

"켜기"와 "반전"의 차이. 켜기는 지금 상태와 상관없이 1로 **정하는** 것이고(전등 스위치를 "켜짐" 쪽으로 미는 것), 반전은 지금 상태를 **뒤집는** 것이다(누를 때마다 바뀌는 버튼식 스위치). 그래서 상태를 기억하는 플립플롭 `lamp` 하나와, 받은 바이트로 다음 상태를 고르는 `case` 문이면 된다.

$$lamp_{next} = \begin{cases} 0 & rx = \text{0x30 ('0')} \\ 1 & rx = \text{0x31 ('1')} \\ \overline{lamp} & rx = \text{0x32 ('2')} \\ lamp & \text{그 밖 (0x01, 'x' 등)} \end{cases} \qquad (\text{rx_valid} = 1\text{인 클록에만})$$

LED 배치. LED1(`led[7]`) = 명령으로 제어하는 램프, LED2 ~ 8(`led[6:0]`) = 마지막으로 받은 바이트의 아래 7비트. 글자 '1'은 `011 0001`, 숫자 1은 `000 0001` 로 켜져서 둘의 차이가 눈으로 보인다. MCU 쪽은 과제6과 같은 전달 코드에 어떤 명령인지 이름만 함께 출력한다(판단은 FPGA가 한다).

③ 포트 · 보드 매핑

과제6과 같은 회로·핀·비트스트림(`LAB6-04_uart_cmd.bit`)을 쓴다. LED1 = L4, LED2~8 = M4 M2 N7 M7 M3 M1 N5.

④ 핵심 코드

MCU · polling_routine + 명령 이름

```
/* STM32_2_과제7: PC -> MCU -> FPGA
 * MCU는 과제6처럼 받은 바이트를 FPGA로 전달하고, 어떤 LED 명령인지 함께 출력합니다.
 * LED 제어 판단은 FPGA가 합니다: '0'(0x30) 끄기, '1'(0x31) 켜기, '2'(0x32) 반전, 그 밖은 유지.
 */
```

```
static const char *led_command_name(uint8_t data)
{
    switch (data)
    {
    case '0': return "LED 끄기";
    case '1': return "LED 켜기";
    case '2': return "LED 반전";
    default:  return "명령 아님(유지)";
    }
}

void polling_routine(void)
{
    int ch = getchar();
    if (ch != EOF)
    {
        uint8_t data = (uint8_t)ch;
        HAL_StatusTypeDef result = HAL_UART_Transmit(&huart3, &data, 1U, 100U);
        printf("명령 0x%02X -> %s  전송 결과: %d\r\n",
               (unsigned int)data, led_command_name(data), (int)result);
    }
}
```

FPGA 쪽 명령 해석은 실험 6 ④의 `uart_cmd_top.v` 중 `case (rx_data)` 블록이다.

⑤ 시뮬레이션 (공동 시뮬레이션)

MCU 모의 실행 입력 `1 1 1 2 2 2 0 x 0 (0x01)` → UART3 바이트 10개 → FPGA TB. TB는 바이트마다 참조 모델(위 식)과 LED1을, 받은 바이트와 LED2~8을 비교하고, 되돌아온 바이트도 검사한다.

MCU VCP 출력 (모의)

```
명령 0x31 -> LED 켜기  전송 결과: 0
명령 0x31 -> LED 켜기  전송 결과: 0
명령 0x31 -> LED 켜기  전송 결과: 0
명령 0x32 -> LED 반전  전송 결과: 0
명령 0x32 -> LED 반전  전송 결과: 0
명령 0x32 -> LED 반전  전송 결과: 0
명령 0x30 -> LED 끄기  전송 결과: 0
명령 0x78 -> 명령 아님(유지)  전송 결과: 0
명령 0x30 -> LED 끄기  전송 결과: 0
명령 0x01 -> 명령 아님(유지)  전송 결과: 0
```

FPGA TB (XSim)

```
2000.2 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
3406.5 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
4812.7 us  PC(FPGA COM) <- 0x31   LED1=1  LED2~8=0110001
6219.0 us  PC(FPGA COM) <- 0x32   LED1=0  LED2~8=0110010
7625.2 us  PC(FPGA COM) <- 0x32   LED1=1  LED2~8=0110010
9031.5 us  PC(FPGA COM) <- 0x32   LED1=0  LED2~8=0110010
10437.7 us  PC(FPGA COM) <- 0x30   LED1=0  LED2~8=0110000
11844.0 us  PC(FPGA COM) <- 0x78   LED1=0  LED2~8=1111000
13250.2 us  PC(FPGA COM) <- 0x30   LED1=0  LED2~8=0110000
14656.5 us  PC(FPGA COM) <- 0x01   LED1=0  LED2~8=0000001
PASS uart_cmd_top: 10 bytes echoed, LED1 follows 0/1/2 commands
$finish called at time : 15333500030 ps : File "C:/Users/UOS/Documents/lab6/lab6_04_uart_cmd/sim/tb.sv"
Line 46
```

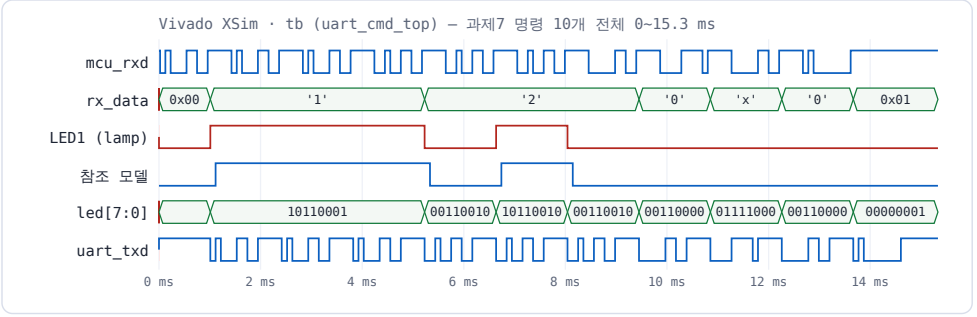


그림 12. 명령 10개. '1' 세 번 동안 LED1은 계속 1, '2' 세 번에 0 → 1 → 0으로 매번 반전, '0'에 0, 'x'(0x78)와 숫자 1(0x01)에는 그대로. 참조 모델과 매 바이트 일치.

⑥ 빌드 결과

대상	결과
task7 (MCU)	text 27,820 B · BUILD OK · 경고 0
FPGA	실험 6과 같은 비트스트림

⑦ 보드 실습 계획

1. `mcu6 task7 -Flash` (FPGA는 과제6 상태 그대로).
2. `talk6 '111222' -Both` → LED1: 켜짐 · 켜짐 · 켜짐 → 꺼짐 · 켜짐 · 꺼짐. 각 단계를 영상에 담는다.
3. `talk6 '0x0' -Both` → 꺼짐 유지. Serial Monitor의 16진수 보내기로 0x01을 보내 LED1은 그대로이고 LED8만 켜지는지(000 0001) 확인 → '1'(0x31)과 숫자 1 구분.

고찰

1. **"TB PASS"와 "보드에서 동작"은 다르다.** 세 FPGA 실습 모두 수업 TB를 통과했지만, 보드 조건을 넣자 서보는 타이밍 위반(−0.609 ns), DAC는 쓰기 폭 여유 0, SRAM은 데이터 홀드 경쟁이 드러났다. 공통 원인은 **TB가 확인하는 것과 보드가 요구하는 것의 차이**다. 느린 클록(서보), 축소된 기준값(DAC 9 ns), 실제와 다른 저장 시점(SRAM 모델)이다. LAB5 ADC에서 배운 대로 외부 칩은 데이터시트 숫자를 모델에 넣어 검사했고, 그래서 실험 전에 고칠 수 있었다.
2. **두 시뮬레이터를 쓰는 이유.** 측정 숫자는 Icarus와 XSim이 모두 같았지만, XSim은 Icarus가 넘어간 두 가지를 잡았다. ① 선언하기 전에 쓴 신호(`wr_active`) ② 시간 0에 X → 0으로 정해지는 에지를 모델이 "쓰기 끝"으로 센 것. 둘 다 회로가 아니라 내가 쓴 TB 모델의 실수였다. 시뮬레이터 하나만 쓰면 그 도구의 너그러움에 기대게 된다.
3. **"같은 순간"은 설계가 아니다.** SRAM 경쟁은 지연 0 시뮬레이션에서 처리 순서로 결과가 갈린다. 보드에서는 나노초 단위의 핀 지연이 같은 역할을 한다. 순서가 결과를 바꾸는 곳에는 클록으로 명확한 간격(여기서는 40 ns)을 두는 것이 안전하다.
4. **곱셈·나눗셈 한 줄의 값.** 서보의 `/1000`은 LAB5 피아노의 변수 나눗셈처럼 큰 회로가 됐고, 이번에는 타이밍까지 어겼다. 단위를 "µs당 클록 수"로 바꿔 생각하니 상수 곱 하나로 끝났다(LUT 782 → 411, 여유 +12.86 ns).
5. **UART에는 확인 응답이 없다.** MCU의 전송 결과 0은 보내기를 마쳤다는 뜻일 뿐이다. 과제6에서 FPGA가 받은 값을 다른 COM 포트로 돌려보내게 한 것은, 받는 쪽의 결과로만 통신 성공을 판단할 수 있기 때문이다. 또 문자 '1'(0x31)과 숫자 1(0x01)이 다르다는 점을 LED2~8로 직접 보이게 했다.
6. **getchar 방식의 한계.** 메인 루프가 `getchar`에서 기다리는 동안 다른 일을 못 하고, printf(9600 bps에서 40글자 ≈ 42 ms) 중에 다음 문자가 오면 놓칠 수 있다. 그래서 당일에는 문자를 하나씩 보낸다(`talk6`는 0.3 s 간격). 계속 받아야 한다면 UART 인터럽트 수신(`HAL_UART_Receive_IT`)이 맞다.
7. **모델의 한계.** SRAM · DAC 모델은 데이터시트의 디지털 시간 조건만 담았다. 보드 배선 지연, DAC 출력의 정착 시간과 잡음, 서보의 실제 각도는 실험 후 보고서에서 오실로스코프와 영상으로 비교한다.

참고문헌

1. [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2013. ISBN 978-0-12-394424-5. — Ch. 3.5 타이밍(셋업·홀드, 슬랙), Ch. 4 HDL의 삼상태 버퍼, Ch. 8.6 I/O의 UART · PWM · DAC.
2. [2] Analog Devices, *AD7302: 2.7 V to 5.5 V, Parallel Input, Dual Voltage Output 8-Bit DAC*, Data Sheet. — Timing Characteristics t_1~t_{10}(WR 폭 20 ns, 데이터 셋업 15 ns, 홀드 4.5 ns), LDAC LOW일 때 WR 상승에서 갱신.
3. [3] Integrated Silicon Solution, Inc., *IS61WV25616ALL/BLL: 256K × 16 High-Speed Asynchronous CMOS Static RAM*, Data Sheet, Rev. E, 2007. — -10 등급 읽기·쓰기 시간 조건(t_{AA} 10, t_{DOE} 4.5, t_{PWE1} 8, t_{SD} 6, t_{HD} 0 ns), "CE · UB/LB · WE 중 하나가 비활성이 되면 쓰기 종료".
4. [4] STMicroelectronics, *stm32g4xx-hal-driver*, GitHub, https://github.com/STMicroelectronics/stm32g4xx-hal-driver, 태그 v1.2.7 (스타 53, 확인일 2026-10-17). — `stm32g4xx_hal_uart.c`의 `HAL_UART_Transmit` · `HAL_UART_Receive`, `stm32g4xx_hal_def.h`의 `HAL_StatusTypeDef`.

수업 자료(인용 출처 수에 넣지 않음): Glaysia/ece2_474 v1.1.0의 docs/STM32_2.md · docs/FPGA_additional.md · docs/C_basics.md, 실습 교재 「Combo II DLD 장비를 이용한 디지털 논리 회로 설계」 p.625(DAC 핀표). LAB3-07 UART 에코 회로(본인 저장소).